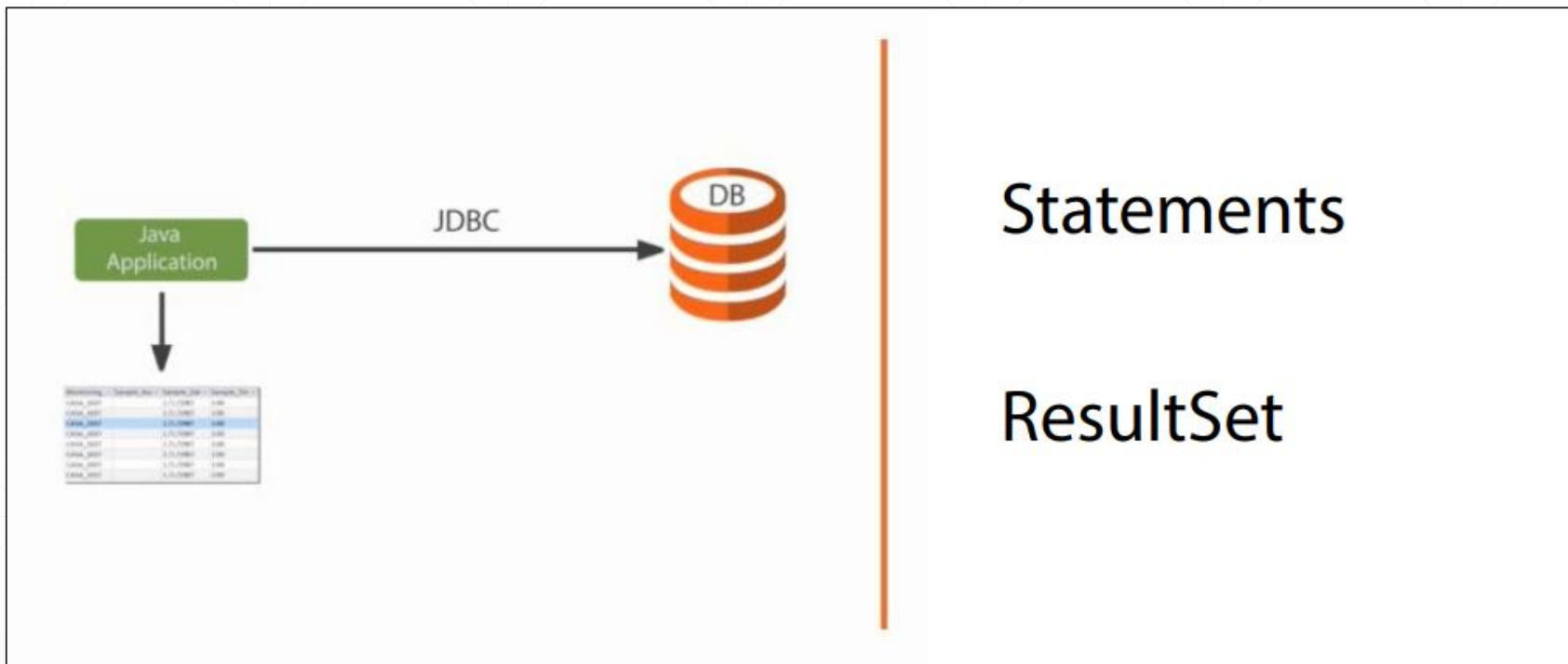




使用JDBC查询数据

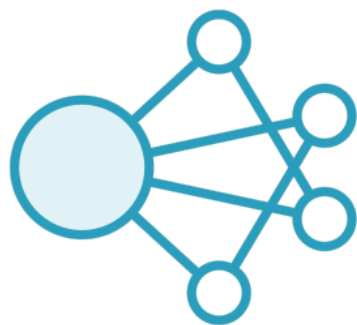
北京理工大学计算机学院
金旭亮

概述



使用JDBC访问数据库的核心类就两个，Statement用于执行SQL命令，ResultSet用于封装从数据库提取回来的记录集。

Connection接口负责创建Statement对象



Connection对象

`createStatement()`



Statement

`prepareStatement(String sql)`



PreparedStatement



Statement对象负责执行无参数的SQL命令。
PreparedStatement对象负责执行有参数的SQL命令。

执行SQL语句



得到一个Statement对象之后，就可以使用该对象的executeQuery()方法来执行数据库查询语句，它返回一个ResultSet对象，包含了查询执行的结果。

```
ResultSet rs = statement.executeQuery("SQL命令");
```



一个ResultSet对象包含了执行某个SQL语句后满足条件的所有的行，它还提供了访问这些行的手段，开发者可以通过一组get方法来访问当前行的不同列。

Statement接口

Statement接口用于执行静态的SQL语句，静态的SQL语句在执行时不需要接收任何参数。其主要功能是将SQL语句传送给数据库，并将SQL语句的执行结果返回。主要包容以下两个方法：

1

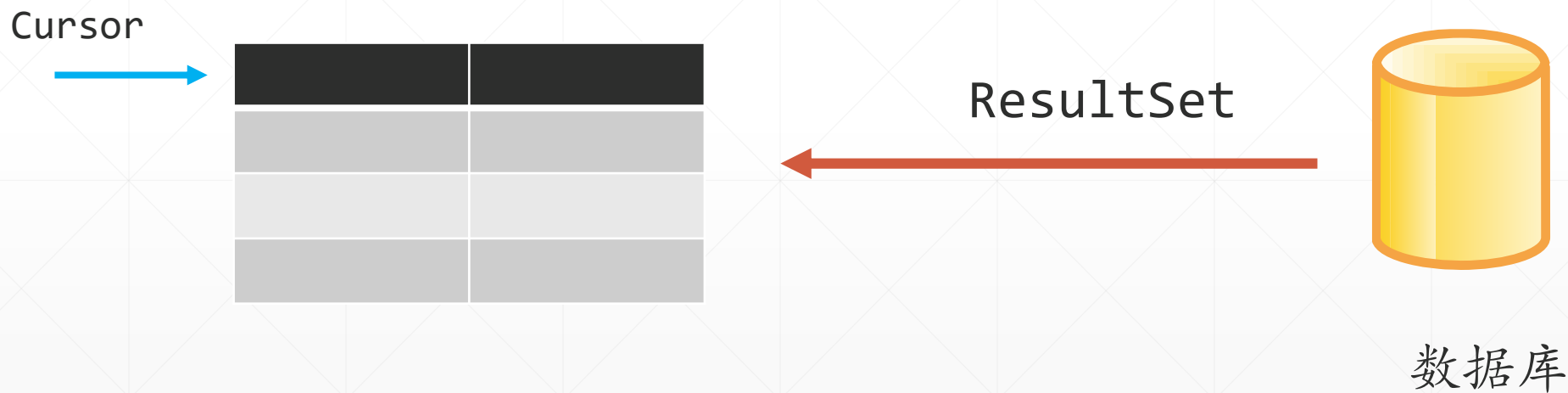
ResultSet executeQuery(SQL命令)：用于执行返回记录集的SQL命令，主要指select命令。

2

int executeUpdate(SQL命令)：用于执行不返回记录集的SQL命令，比如delete和update命令，返回值表示影响的记录条数。

使用ResultSet

ResultSet封装了从数据库中返回的记录集，它有一个读写指针（称为“Cursor”），指向当前正在处理的行，需要时，它可以前后移动.....



使用ResultSet对象



ResultSet对象内部拥有的“读写指针”，称为“**cursor**（游标）”。刚开始时，**cursor**指向第一条记录“前面”的位置，可使用ResultSet对象的**next()**方法移动指针，从而一条条地读取记录。

```
// 查询表中保存的数据
rs = statement.executeQuery("select * from OrderClient");
// 循环输出查询到的数据
while (rs.next()) {
    System.out.println("-----");
    System.out.println("ClientID = " + rs.getInt("ClientID"));
    System.out.println("ClientName = " + rs.getString("ClientName"));
    System.out.println("Address=" + rs.getString("Address"));
}
```

读取当前行



ResultSet提供了getXXX()系列方法用于读取当前行中的每个字段的值。

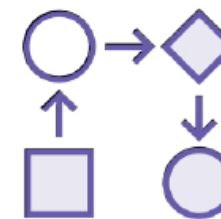


字段的选择有两种方式，一种是指定它的位置索引（这个位置是由SQL命令或数据库表结构决定的），另一种是按照字段的名字访问。



由于数据库表的字段个数有可能增减，顺序可能变化也比较频繁，所以，推荐使用名字来访问字段值。

四步从数据库中提取数据



建立与数据库管理系统的连接



创建Statement对象，封装SQL命令



调用Statement相关的方法执行SQL命令



处理ResultSet：循环访问每一行

```

private static void testConnectionAndQuery() {
    Connection connection = null;
    try {
        // 创建数据库连接
        connection = DriverManager.getConnection( url: "jdbc:sqlite:test.db");
        // 准备创建操作与存取的数据库命令
        Statement statement = connection.createStatement();
        // 查询表中保存的数据
        ResultSet rs = statement.executeQuery( sql: "select * from OrderClient");
        while (rs.next()) {
            // 输出查询到的数据
            System.out.println("-----");
            System.out.println("ClientID = " + rs.getInt( columnLabel: "ClientID"));
            System.out.println("ClientName = " + rs.getString( columnLabel: "ClientName"));
            System.out.println("Address=" + rs.getString( columnLabel: "Address"));
        }
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    } finally {
        try {
            if (connection != null)
                connection.close();
        } catch (SQLException e) {
            System.err.println(e);
        }
    }
}

```

SQL 命令由Statement对象承载，执行之后，ResultSet保存从数据库中传回的数据

ResultSet提供了多种方法读取当前记录的字段值

数据提取完毕之后，需要关闭所有数据库相关的对象

SQLiteJDBCTest.java

多学一招：限制一次读取的数量



Statement对象有一个方法可以设置一次最多读取的记录数：

```
Statement stmt = conn.createStatement();  
stmt.setMaxRows(100);
```



但更好的方式是使用SQL命令的Limit子句，它接收两个参数，第一个参数表示起始位置，第二个参数表示要读取多少条记录：

```
SELECT * FROM test Limit 5,5
```

关闭创建的各个对象



一个Statement对象在同一时间只能打开一个结果集，所以如果在同一个Statement对象中运行下一条SQL语句时，第一条SQL语句生成的ResultSet对象就被自动关闭了。



需要时，也可以通过调用ResultSet接口的close()方法来手工关闭。



关闭Statement对象和Connection对象需要分别使用各自的close()方法。

实际开发中经常这样写JDBC代码

使用JDK 7引入的try with closable resource特性，可以简化JDBC中关闭相关对象的代码：

```
try (  
    Connection conn = ...;  
    Statement stmt = conn.createStatement(...);  
    ResultSet rs = stmt.executeQuery(...);  
) {  
    //...  
} catch (SQLException e) {  
    //...  
}
```



不再需要在finally块中写一堆的close相关语句。



Spring JDBC Template提供了更有效的封装。

可前后移动的ResultSet



默认情况下，一些数据库返回的ResultSet是仅向前的，但大多数数据库能够返回支持前后移动读取数据的ResultSet，可以使用以下方式保证获得可移动的结果集：

```
Statement stmt =  
    conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,  
                          ResultSet.CONCUR_READ_ONLY);
```



默认情况是指createStatement()方法中不包含任何参数。

在可滚动ResultSet中前后移动

Cursor



`beforeFirst()`

`afterLast()`

`first()`

`last()`

`previous()`

`next()`

ResultSet的类型



Type_Forward_Only: 仅向前



Type_Scroll_Insensitive: 可前后移动，对数据变动不敏感



Type_Scroll_Sensitive: 可前后移动，对数据变动敏感



SQLite只支持仅向前的ResultSet，SQL Server等功能更强的数据库，支持返回可前后滚动的记录集。

动态查询数据库



在使用Statement对象时，每次查询都需要将SQL语句传递给数据库执行，当多次执行同一条SQL语句时，每次都需要重复相同的工作，对效率有影响，此时可以考虑使用PreparedStatement对象。



PreparedStatement多用于执行**动态**的SQL语句。动态的SQL语句，可以在SQL语句中定义参数，这可以大大地提高程序的灵活性和执行效率。

PreparedStatement基本使用技巧



PreparedStatement封装的，是带有参数的SQL命令，参数使用“?”代表。



一条SQL命令可以定义多个参数，这些参数按照出现位置，对应着一个从1开始的位置索引。



PreparedStatement提供了setXXX（位置索引，值）这样的方法来动态地设定要执行的SQL的参数值。

//测试执行带参数的SQL命令

```
private static void testPreparedStatement(int ClientID) {  
    Connection connection = null;  
    try {  
        // 创建数据库连接  
        connection = DriverManager.getConnection("jdbc:sqlite:test.db");  
        String sql = "select * from OrderClient where ClientID=?";  
        // 准备创建操作与存取的数据库命令  
        PreparedStatement statement = connection.prepareStatement(sql);  
        //设置参数值, 第一个参数指的是SQL命令中的参数位置(从1开始, 顺序递增), 第二个参数是SQL参数的具体值  
        statement.setInt(parameterIndex: 1, ClientID);  
        // 查询表中保存的数据  
        ResultSet rs = statement.executeQuery();  
        while (rs.next()) {  
            // 输出查询到的数据  
            System.out.println("-----");  
            System.out.println("ClientID = " + rs.getInt(columnLabel: "ClientID"));  
            System.out.println("ClientName = " + rs.getString(columnLabel: "ClientName"));  
            System.out.println("Address=" + rs.getString(columnLabel: "Address"));  
        }  
    } catch (SQLException e) {...} finally {...}  
}
```

用问号定义查询
参数

处理JDBC异常-1



当程序运行出现错误时，JDBC API通常会抛出一个`SQLException`异常，此异常派生自`Exception`，但添加了以下新的方法：

- ✓ `getErrorCode()`：一个整数，代表出现的错误类别
- ✓ `getSQLState()`：字符串，封装出错状态信息



在实际运行时，JDBC抛出的异常可能不止一个，这时，可以使用`getNextException`，`setNextException`和`iterator`访问到所有抛出的异常并处理它们。

处理JDBC异常-2



连接不同的数据库，当程序代码有错时，抛出的异常是不一样的。

例如，如果发送到数据库的SQL命令有错，不同厂家开发的数据库，会抛出不同类型的异常，这些异常，通常都是SQLException的子类.....

数据库类型	抛出的异常类型
HSQL	<code>java.sql.SQLSyntaxErrorException</code>
MySQL	<code>com.mysql.jdbc.exceptions.jdbc4.MySQLSyntaxErrorException</code>
SQLite	<code>org.sqlite.SQLiteException</code>
.....	

处理JDBC异常-3



以下代码可以显示底层数据库抛出的异常，但往往需要查询具体的数据库文档，才能了解其给出的各项信息的含义。

```
public static void processException(SQLException e) {  
    System.err.println("Error message:" + e.getMessage());  
    System.err.println("Error code:" + e.getErrorCode());  
    System.err.println("SQL state:" + e.getSQLState());  
}
```